Decision Trees: Recursive Partitioning, Information Gain, and Pruning

Lecture 27

CS 4880

Kirk Duran

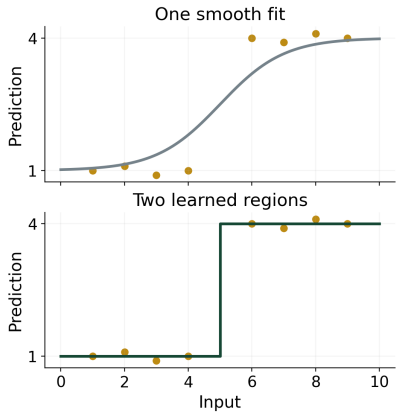
From Smooth Fits to Explicit Decisions

Sometimes the world behaves more like a collection of rules than one smooth trend.

- Lecture 26 fit lines, curves, and logistic scores to labeled outcomes.
- Printer failure risk may change abruptly above a job-load threshold; temperature may change the rule for lighter jobs.
- A tree learns **which question to ask next** and makes separate predictions in different regions.

Key idea

A tree can model sharp thresholds and interactions without adding polynomial features by hand.



Akinator: The Value of the Next Question

An informative question separates the remaining possibilities into useful groups.

- Akinator provides an intuition: some questions eliminate more possibilities than others.
- "Is it fictional?" can be more useful than "Does it wear a hat?" depending on the candidates still remaining.
- A decision tree learns feature questions from labeled examples. The illustration is an analogy, not a description of Akinator's implementation.

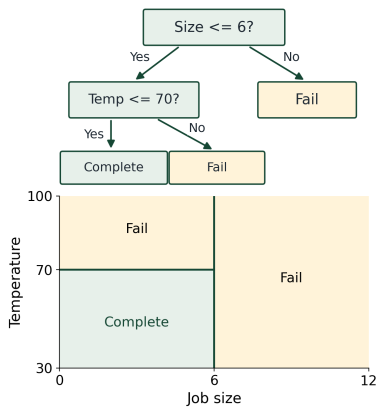


Anatomy of a Decision Tree

Prediction means following answers from the root until a leaf supplies a label.

- **Root:** the first test applied to every example.
- **Internal node:** another feature-based question.
- **Branch:** one answer to that question.
- **Leaf:** final prediction; for classification, usually the local majority class.

One root-to-leaf path is an **if–then rule**. The split question and predicted label play different roles.



Recursive Partitioning Creates Decision Boundaries

Every test separates the observations; later tests refine only one resulting group.

- **Categorical feature:** "Warm $=$ Yes?" produces groups based on feature values.
- **Numeric feature:** "Job size ≤ 6?" puts values on opposite sides of a threshold.
- Multiple tests create rectangular, *axis-aligned* regions when plotted against two numerical features.
- Interactions arise from *the path*: the temperature question can matter only after job size was tested.

One path as a compact rule

$$(\text{Job size} \leq 6) \ \wedge \ (\text{Temperature} \leq 70) \quad \Longrightarrow \quad \widehat{y} = \text{Complete}.$$

$\wedge$ means "and." The tree represents this combination by following two branches, not by computing an explicit product of features.

What Makes One Split Better Than Another?

A helpful question creates groups whose observed labels mostly agree.

- A **pure group** contains one class only: no disagreement about its training labels.
- A **mixed group** contains both failed and complete observations.
- Many tiny branches can appear pure on training data without predicting well. Validation and pruning help judge whether the split is useful.
- Large and small children must not contribute equally: weight by how many examples each contains.

Compare two possible questions

Good: 5 failed / 0 complete versus 1 failed / 4 complete.

Weak: 4 failed / 1 complete versus 2 failed / 3 complete.

Entropy Measures Uncertainty in a Group

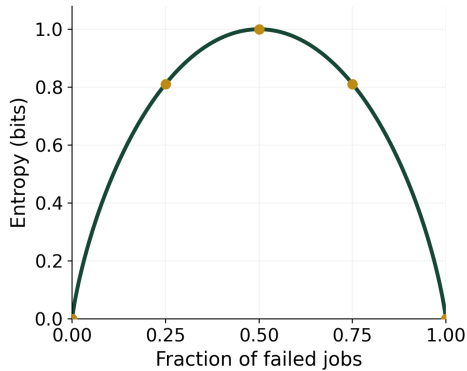
Entropy is low when the answer is nearly certain and high when classes are balanced.

- All failed or all complete: **zero entropy**.
- Half failed and half complete: **highest binary entropy**, one bit.
- A 75% / 25% split is uncertain, but less uncertain than 50% / 50%.

Binary entropy: notation for this intuition

$$H(p) = -p \log_2(p) - (1-p) \log_2(1-p).$$

p is the fraction of failed examples; $1-p$ is the complete fraction. Treat $0 \log_2 0$ as zero.



A Small Dataset We Can Trace by Hand

Predict whether a printer job fails using two inputs recorded before starting.

Job	Load	Warm?	Observed outcome
1	High	Yes	Fail
2	High	Yes	Fail
3	High	Yes	Fail
4	High	No	Fail
5	High	No	Fail
6	Low	Yes	Fail
7	Low	Yes	Complete
8	Low	No	Complete
9	Low	No	Complete
10	Low	No	Complete

The label is the outcome; Load and Warm are features. This synthetic example has six failures and four completions. Jobs 6 and 7 have identical recorded inputs.

Calculate Uncertainty at the Root

Start by counting labels. The ten printer jobs contain six failed and four complete.

Group	Failed (Y)	Complete (N)	Total
All observations	6	4	10

- The failed fraction is $6/10$, so the group is mixed rather than pure.
- We need a score for this *starting uncertainty* before evaluating questions.

Substitute the two class fractions

$$H(S) = -\frac{6}{10}\log_2\left(\frac{6}{10}\right) - \frac{4}{10}\log_2\left(\frac{4}{10}\right) \approx \boxed{0.971 \text{ bits}}.$$

The logarithms convert class proportions into an uncertainty score; the main procedure is count $\rightarrow$ fraction $\rightarrow$ entropy.

Information Gain: How Much Uncertainty Did the Question Remove?

Subtract the uncertainty left after the split from the uncertainty we had before it.

1. Calculate the parent group's entropy.
2. Split the records using a candidate question.
3. Calculate entropy inside each child and weight by its share of the records.
4. Prefer the question producing the **largest reduction**.

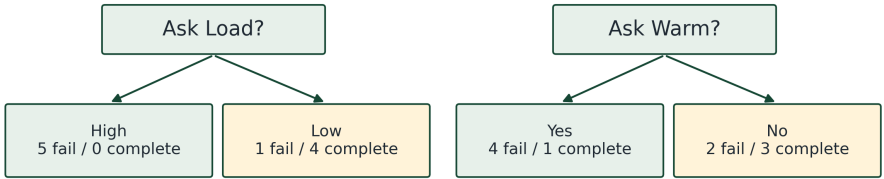
One formula for all candidate splits

$$\underbrace{IG(S,X)}_{\text{gain}} = \underbrace{H(S)}_{\text{before}} - \underbrace{\sum_{v} \frac{|S_v|}{|S|} H(S_v)}_{\text{weighted uncertainty after}} .$$

S_v is the group receiving answer v; $|S_v|/|S|$ is its fraction of observations. The sum simply adds the child contributions.

Worked Split: Ask About Load or Warm?

Load is the better root question: its children agree with their labels more strongly.



Compare both reductions from the same root entropy of 0.971 bits

$$H_{\text{after}}(\text{Load}) = \tfrac{5}{10}H(1) + \tfrac{5}{10}H(\tfrac{1}{5}) = 0.361, \qquad IG = 0.610,$$
$$H_{\text{after}}(\text{Warm}) = \tfrac{5}{10}H(\tfrac{4}{5}) + \tfrac{5}{10}H(\tfrac{2}{5}) = 0.846, \qquad IG = 0.125.$$

Here $H(p)$ is binary entropy. The bigger gain leaves less uncertainty, so **Load becomes the root**.

Build the Tree by Repeating the Same Decision

Use the best local question, divide the records, then solve the smaller problems.

1. **Stop** if labels are pure or a growth rule says to stop; make a leaf.
2. **Search** candidate feature tests and compute each gain.
3. **Choose** the largest useful gain and partition the data.
4. **Recurse** separately on each child group.

Key idea

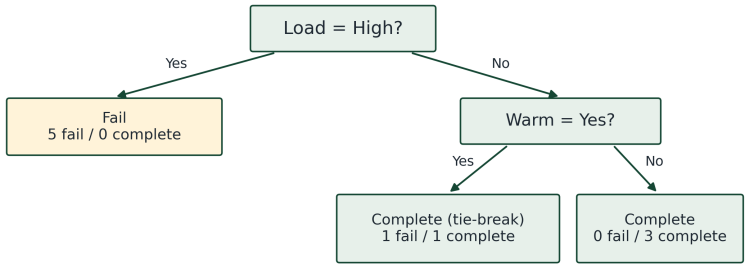
"Greedy" means best split *now*. It does not guarantee the best complete tree; insisting on immediate gain can miss interactions that need another question.

Readable recursive pseudocode

```
BuildTree(records):
  if stop: return majority leaf
  find best candidate split
  if no gain: return majority leaf
  left = BuildTree(left records)
  right = BuildTree(right records)
  return node(test, left, right)
```

Continue the Example: A Small Complete Tree

Only the mixed low-load branch needs another question.



- **High load:** all five jobs failed; stop immediately.
- **Low load:** ask Warm. With No, all three complete; with Yes, one fails and one completes.
- Break the tied leaf using its parent's majority (complete). One training error remains because the recorded inputs cannot separate these jobs.

Why Can the Tree Still Make a Mistake?

A decision rule cannot distinguish observations that look identical to the model.

The inseparable pair in our data

Jobs 6 and 7 both have **Load = Low** and **Warm = Yes**. One failed; the other completed.

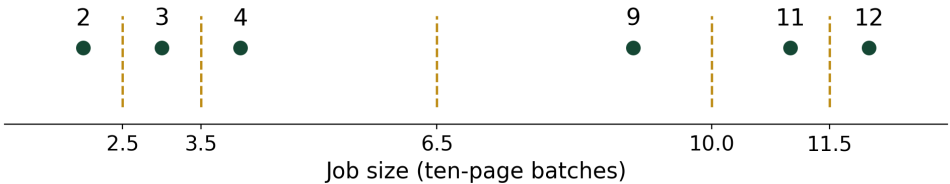
- Every deterministic tree using only these features sends both jobs down the same path.
- More splitting cannot recover a distinction that the inputs never recorded.
- Omitted sensors, random effects, noisy labels, or unrecorded conditions may explain the different outcomes.

The limitation in compact form

$$\mathbf{x}_6 = \mathbf{x}_7 \quad \Longrightarrow \quad f(\mathbf{x}_6) = f(\mathbf{x}_7), \quad \text{even though } y_6 \neq y_7.$$

Numerical Features: Where Should We Place the Cutoff?

For a numerical feature, search possible "less than or equal to" cutoffs.



1. In a separate six-job example, sizes are $2, 3, 4, 9, 11, 12$ ten-page batches.
2. Candidate midpoints are $2.5, 3.5, 6.5, 10, 11.5$.
3. For each midpoint, split the records and measure the impurity left behind.
4. Choose the feature and threshold giving the greatest impurity reduction.

A Numeric Split Is the Same Impurity Calculation

A threshold simply determines who goes left and right; the split score works as before.

- Candidate: "Job size ≤ 6.5?"
- Place 2, 3, 4 on the left; 9, 11, 12 on the right.
- If every left observation is complete and every right observation is failed, both children are pure.
- Trying all midpoints is a search over possible decision rules, not advanced calculus.

Weighted child impurity

$$I_{\text{split}}(t) = \frac{n_L}{n} I(L) + \frac{n_R}{n} I(R).$$

Smaller is better. Here each child is pure, so the score is zero.

Key idea

The same mechanism works for classification (entropy or Gini) and regression (e.g., squared-error reduction).

Stop Before a Tree Memorizes the Training Set

Depth and leaf-size limits prevent the algorithm from inventing a rule for every tiny exception.

`max_depth`	`min_samples_leaf`
Maximum number of splits along a path. Smaller values produce shallower, simpler trees.	Minimum number of training examples permitted in each leaf. Prevents tiny, fragile groups.

- These are **pre-pruning** controls: they constrain growth before the tree is finished.
- Too restrictive $\rightarrow$ meaningful patterns are missed (underfitting).
- Too flexible $\rightarrow$ sample-specific noise can become rules (overfitting).

Python Example 1: California Housing Regression Tree

The same recursive structure can predict a number rather than a class.

```
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.tree import DecisionTreeRegressor, plot_tree

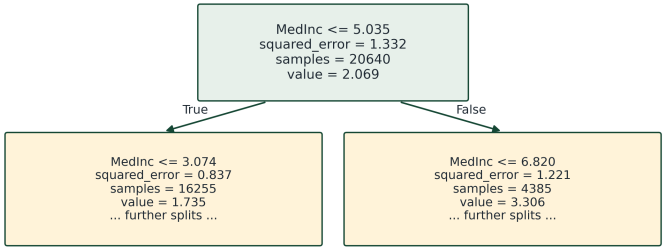
# Load data
data = fetch_california_housing()
X = data.data
y = data.target
feature_names = data.feature_names

# Fit for display; held-out evaluation comes later.
tree = DecisionTreeRegressor(max_depth=4, random_state=42)
tree.fit(X, y)

# Plot the tree
plt.figure(figsize=(20, 10))
plot_tree(tree, feature_names=feature_names,
          filled=True, rounded=True, fontsize=10)
plt.title("Decision Tree Regression on California Housing Dataset")
plt.show()
```

Reading the California Housing Tree

A regression leaf predicts the mean target of its training records.



- Read `MedInc` at the root: True goes left, False right. Top two levels shown; fitted model depth is four.
- `samples` counts records reaching the node; `value` is its mean target.
- `squared_error` summarizes spread of targets within the node; reducing this spread replaces class entropy as the regression goal.

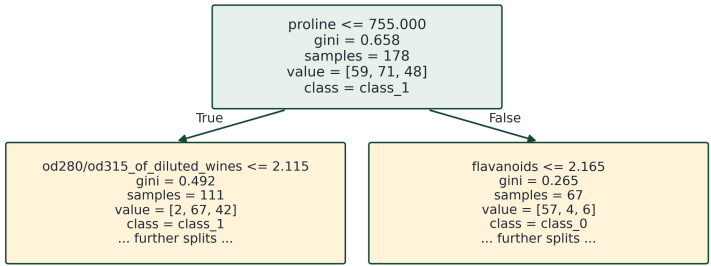
Python Example 2: Wine Classification Tree

Now a leaf predicts a class using the observations that reach it.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.tree import DecisionTreeClassifier, plot_tree
# Load dataset
wine = load_wine()
X = wine.data
y = wine.target
feature_names = wine.feature_names
class_names = wine.target_names
# Fit for display; held-out evaluation comes later.
clf = DecisionTreeClassifier(max_depth=4, random_state=42)
clf.fit(X, y)
# Plot the tree
plt.figure(figsize=(20, 10))
plot_tree(clf, feature_names=feature_names,
          class_names=class_names, filled=True,
          rounded=True, fontsize=10)
plt.title("Decision Tree on Wine Dataset")
plt.show()
```

Reading the Wine Classification Tree

Read each node's question, sample count, class distribution, and predicted class.

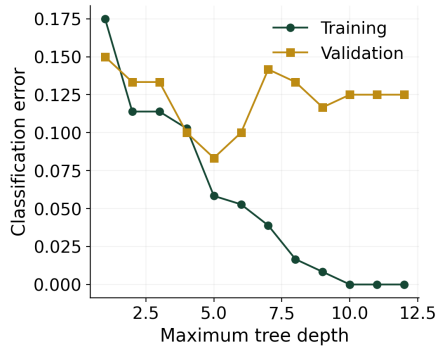


- `value` shows class counts; `class` is the majority. Two levels shown; fitted model depth is four.
- `gini` measures impurity; Gini is scikit-learn's default criterion.
- `criterion="entropy"` uses information gain instead. This model keeps the default Gini criterion.

A Deep Tree Can Fit the Wrong Details

Perfectly matching old observations does not mean predicting new jobs well.

- Deep trees can carve out tiny regions to explain isolated training examples or misreadings.
- **Training error** often falls as complexity grows; **validation error** may stop improving or increase.
- Smaller trees have more bias (stronger simplifying assumptions); large trees can have more variance (sensitivity to the sample).



Two Ways to Prune: Before Growth or Afterward

Pruning trades a little training fit for simpler rules that may generalize better.

Pre-pruning	Post-pruning
Stop growth using `max_depth`, `min_samples_leaf`, or a minimum split improvement.	Grow a larger tree, then remove branches when their improvement is too small to justify their complexity.
Easy to enforce, but may block useful later questions.	Cost-complexity pruning is controlled by `ccp_alpha`.

Cost-complexity pruning in one line

$$R_\alpha(T) = R(T) + \alpha\,|\text{leaves}(T)|.$$

$R(T)$ is the training impurity cost; $|\text{leaves}(T)|$ counts final leaves; α is a penalty for extra leaves. Larger α generally favors smaller trees.

Choose the Tree Using Held-Out Evidence

Compare models on held-out examples, not training accuracy alone.

Candidate	Leaves / depth	Training accuracy	Validation accuracy
Unrestricted	50 / 10	100.0%	87.5%
Pre-pruned	9 / 4	88.6%	89.2%
Post-pruned	7 / 5	92.5%	92.5%

- Separate synthetic experiment: 360 training, 120 validation, 120 test cases. Training fits rules; validation compares depth and pruning settings.
- Here the frozen post-pruned model scores 88.3% on untouched test cases.
- If failures are rare, also examine missed failures and false alarms, not just accuracy.

Key idea

Removing a special-case leaf can make some training labels wrong but improve predictions on new jobs.

Explain a Decision, Then Examine Its Failure

The tree gives an inspectable chain of reasons; it cannot reveal what the sensors never measured.

Two readable rules from our printer tree

Rule 1: IF load is high, THEN predict failed.

Rule 2: IF load is low AND warm is no, THEN predict complete.

- Trace a misclassification through its nodes: which test routed it to this leaf?
- Check the leaf's sample count: a rule based on one or two cases may be fragile.
- Missing features might include paper quality, component wear, or sensor confidence.
- Understandable rules help debugging, auditing, and explaining an agent's action to a human operator.

Synthesis: From Questions to Generalization

1. **Representation:** decision paths form conditional if–then rules and nonlinear, axis-aligned partitions.
2. **Learning:** entropy describes class uncertainty; information gain chooses a useful local question; recursion builds the rest of the tree.
3. **Limitations:** greedy choices are not globally optimal, and contradictory labels with identical features cannot be perfectly separated.
4. **Generalization:** numeric thresholds, depth limits, leaf-size controls, pruning, and held-out validation determine whether those rules transfer.

Key idea

A good decision tree asks informative questions *and* knows when to stop asking.